Data Analysis with Python - Mock Midterm Answer Key

Units 2-5 | 60 minutes | Total 100 pts (MCQ 60 + Free Response 40)

Part 1 - Multiple Choice

Answer key and one-line explanations.

Q	Unit	Answer	Explanation
1	Unit 2	C	The comprehension keeps 6, 6, 7 and 8 (4 values); ratings equal to 5 are excluded because the condition is r > 5.
2	Unit 2	D	Adelie appears twice in the list, so c["Adelie"] is 2.
3	Unit 2	B	sum(int(r) for r in rings) converts each string to int inside a generator before summing; the others sum strings and fail or give nonsense.
4	Unit 2	A	zip pairs each name with its mpg, the f-string formats each pair, and " | ".join(...) merges them into a single string.
5	Unit 2	B	800 < 1000 but not < 500, so the elif branch returns "mid".
6	Unit 3	D	mean = (1.4 + 4.7 + 5.1 + 1.3 + 5.9) / 5 = 3.68.
7	Unit 3	B	The boolean mask is True for 0.35, 0.53 and 0.42; sum counts the True values: 3.
8	Unit 3	C	np.sort(mpg) sorts ascending; [-3:] takes the last three entries, i.e. the three largest. np.max takes one value, [:3] gives the smallest, np.unique removes duplicates.
9	Unit 3	A	reshape(3, 4) keeps the same 12 elements and returns the shape (3, 4).
10	Unit 3	B	A Generator created with a fixed seed is reproducible; choice(344, ...) draws indices from 0-343 without replacement. A draws 5 values from only {0..4}.
11	Unit 4	C	value_counts() returns a Series indexed by species with the row counts, sorted from most to least frequent.
12	Unit 4	A	loc[boolean_mask] keeps the rows where the mask is True. The other forms mix columns into the mask, use invalid tuple syntax, or expect column names.
13	Unit 4	D	groupby("island") splits rows by island, then ["body_mass_g"].mean() averages that column within each group.
14	Unit 4	C	Sort descending by flipper length and take the first 5 rows; ascending=False is required so the longest come first.
15	Unit 4	A	0.87 (flipper length vs body mass) is the largest positive value outside the diagonal.
16	Unit 5	B	na_values lists the tokens that should become NaN on load; there is no missing/na/replace parameter in read_csv.
17	Unit 5	D	dropna(subset=["age"]) only checks the age column; plain dropna() would also drop rows with other missing values.
18	Unit 5	D	keep=False removes every duplicated id, so both A1 and A2 rows are dropped; only the unique A3 row remains.
19	Unit 5	A	str.contains returns a boolean Series marking which names contain the substring; that Series can then be used as a filter mask.

20	Unit 5	B	map applies a value -> value dictionary element-wise and yields booleans; astype takes a dtype mapping, not value mappings.

Part 2 - Free Response

Q21 - Preparing the raw data in Python [Unit 2]

Model answer:

```
# (a)
def parse_line(line):
    block_id, prox, value, income, age = line.strip().split(",")
    return block_id, prox, float(value), float(income), int(age)

# (b)
with open("housing_raw.txt") as f:
    blocks = [parse_line(line) for line in f if line.strip()]

# (c)
values = [v for _, _, v, _, _ in blocks]
avg = sum(values) / len(values)
print(f"Average value: ${round(avg, 2)}")


from collections import defaultdict
count = defaultdict(int)
for _, p, _, _, _ in blocks:
    count[p] += 1
print(count["INLAND"])
```

Scoring:

- (a) [2] strip + split + float/int casts in the returned tuple.

- (b) [2] with open(...) block; comprehension condition skips blank lines.

- (c) [3] average = (180000 + 265000 + 95000 + 320000 + 150000) / 5 = $202000.0; INLAND count = 3 via a dict.

Q22 - Analyzing housing values with NumPy [Unit 3]

Model answer:

```
import numpy as np

# (a)  (np.where is also fine)
valid = values[values > 0]
print(round(valid.mean(), 2))

# (b)
print((values > np.mean(values)).sum())
print(values[proximity == "INLAND"].mean())

# (c)
rng = np.random.default_rng(seed=2026)
idx = rng.permutation(len(values))[:3]
print(idx, values[idx])
# A fixed seed makes the sample reproducible for the report/audit.
```

Scoring:

- (a) [2] excludes the 0.0 error: valid mean.

- (b) [3] above-mean count = 4 (180000, 265000, 320000, 240000); INLAND mean computed with a boolean mask.

- (c) [3] default_rng(2026).permutation(len(values))[:3]; fixed seed = reproducible sample.

Q23 - Exploratory analysis with pandas [Unit 4]

Model answer:

```
# (a)
print(df["median_house_value"].describe())
# Facts: mean 206856 > median 179700, so value is right-skewed;
# std 115396 and max 500001 (vs 75% at 264725) indicate high-value
outliers.

# (b)
print(df.loc[df["ocean_proximity"] == "INLAND",
"median_house_value"].mean())

# (c)
median_income (r = 0.69) is the strongest positive correlate:
# block groups with higher median incomes tend to have higher house
values.

# (d)
print(df.sort_values("median_house_value", ascending=False).head(5))
# Most common ocean proximity: <1H OCEAN (420 of 1000).
```

Scoring:

- (a) [3] correct describe() call; two correct facts from the output (e.g. mean vs median skew, outlier range).

- (b) [3] correct filter+mean lines

- (c) [3] median_income (0.69); plain-language interpretation.

- (d) [3] sort_values("median_house_value", ascending=False).head(5); <1H OCEAN most common.

Q24 - Cleaning, and comparing two AI assistants [Unit 5]

Model answer:

```
# (a)
df = pd.read_csv("housing.csv", na_values=["", "-999"])
print(df.isna().sum())

# (b)
df = df.dropna(subset=["median_house_value"])
df = df.drop_duplicates(subset=["block_group_id"], keep="last")

# (c)
df["ocean_proximity"] = df["ocean_proximity"].str.strip()
df["ocean_proximity"] = df["ocean_proximity"].replace({"NEAR BAY":
"Bay", "NEAR OCEAN": "Ocean"})
```

```
# (d)
# (i) Model A is more trustworthy: it strips whitespace in
ocean_proximity,
# so block groups with " <1H OCEAN " are included, and it drops
duplicate
# block_group_ids so each block group is counted once.
# (ii) Model B skipped str.strip() and
drop_duplicates(subset=["block_group_id"]).
# Rows whose proximity has a trailing space are silently excluded,
and duplicate
# ids are double-counted; both move the mean (here 240244.0 ->
235800.0) - a
# plausible-looking wrong number.
# (iii) Verification: compare df.shape before/after each cleaning
step; run
# value_counts() on ocean_proximity before/after strip; reproduce
the mean
# and check min/max with describe().
```

Scoring:

- (a) [3] read_csv with na_values=["", "-999"]; isna().sum() for per-column counts.

- (b) [3] dropna(subset=["median_house_value"]) and drop_duplicates(subset=["block_group_id"], keep="last").

- (c) [3] str.strip() and replace (or map with a fallback) for the two renames.

- (d) [4] (i) Model A + reason (str.strip and drop_duplicates); (ii) the skipped steps named and the direction of the effect (omitted " <1H OCEAN " rows and double-counted duplicate ids move the mean, 240244.0 -> 235800.0); (iii) one verification idea (row counts / value_counts / describe).